MiniOS Technical Documentation

1. INTRODUCTION

MiniOS Omega is a simulated operating system developed using Python. It allows users to interact with the system through a command-line interface by entering commands. It includes features such as file management, process management, memory management, user login, system information, and error handling. Users can create and manage files, simulate processes, monitor memory usage, and access system details. The system also checks login credentials and displays error messages when invalid commands or inputs are entered. MiniOS Omega is designed to demonstrate the basic functions of an operating system in a simple and interactive way. It is organized into five Python files, with each file handling a specific part of the system. This structure makes the program easier to understand, manage, and maintain. The file management feature allows users to create folders, write file contents, and read stored information. Process management uses process IDs to identify and track simulated processes. Memory management shows how memory is allocated and how much remains available. The system also provides a login screen that allows users to access it using built-in accounts. All files, processes, and memory allocations are temporary and disappear when the program closes. Through its features, MiniOS Omega provides a practical example of how basic operating system functions work together. The project helps demonstrate important programming and operating system concepts in a simple environment.

2. Project Objectives

The primary objectives of the MiniOS project are to: Understand OS Mechanics: Gain hands-on experience with how an operating system bridges user commands and system resources.

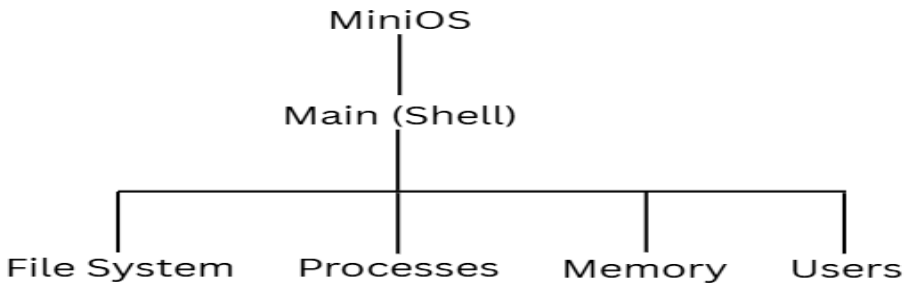
- Implement Core Subsystems: Successfully simulate the four pillars of operating system design: File Systems, Process Management, Memory Allocation, and User Security.

- Develop Modular Software: Practice writing clean, compartmentalized Python code where distinct modules operate independently but communicate through a central shell.

- Handle Exceptions Gracefully: Create a resilient command loop that intercepts errors without crashing the entire system.

3. System Architecture

The architecture follows a centralized hub-and-spoke model. The Main Shell acts as the central router, intercepting user inputs and delegating tasks to four specialized, independent modules.



4. Program Structure

MiniOS is split into five Python files. Each file handles one area of the operating system, and main.py ties them together. Splitting the code this way keeps every module small and lets each one be changed without touching the others.

File	Responsibility
main.py	Entry point, banner, command table, shell loop, general commands
users.py	User accounts and the login screen

filesystem.py	In-memory file and folder tree and its commands
processes.py	Simulated process table and its commands
memory.py	Simulated RAM accounting and its commands

main.py is the file that is run to start the system. It imports the other four modules and does the following:

- **Constants:** OS_NAME and VERSION are used by the banner and the systeminfo command.
- **General commands:** help, clear, date, time, whoami, and systeminfo are defined here because they do not belong to any single subsystem. clear uses cls on Windows and clear on other systems, and date and time use Python's datetime module.
- **Command table:** The COMMANDS dictionary maps each command name to a tuple of (function, description). All 20 commands are registered here, including those imported from the other modules, for example "dir": (filesystem.cmd_dir, "List files and folders"). The help command simply loops over this dictionary, so a new command appears in help automatically once it is added.
- **Shell loop:** shell() repeatedly shows the MiniOS> prompt, reads a line, splits it into a command name (lower-cased) and a list of arguments, looks the name up in COMMANDS, and calls the matching function with the arguments. The loop ends when a command returns the string "EXIT".
- **Calls users.login():** Called from main(). If login succeeds it prints the banner and starts the shell; if it fails the program ends.

Every command function has the same signature, cmd_xxx(args), which receives the list of words typed after the command name. This uniform design is what makes the dispatch table possible.

users.py

Holds the USERS dictionary, the current_user variable, and the login() function.

filesystem.py

Holds the file tree (fs_root), the current directory path (cwd), a helper _current_dir(),

and the commands dir, mkdir, create, write, read, delete, and cd.

processes.py

Holds the process table (processes), the PID counter (next_pid), and the commands run, process, and kill.

memory.py

Holds the memory constants and counter (TOTAL, OS_RESERVED, allocated), the helpers _get_amount() and _used(), and the commands memory, allocate, and free.

5. Execution Flow

1. **User runs python main.py**: main() starts.
2. **main() calls users.login()**: The user has three attempts to log in.
3. **Banner and shell()**: The banner is printed and shell() starts.
4. **Command Parsing**: Each typed line is parsed and dispatched through COMMANDS to the correct module.
5. **Shutdown**: Typing exit (or pressing Ctrl+C / Ctrl+D) shuts the system down.

6. File Management

The MiniOS file system is a virtual file system that exists only in memory. It is implemented with nested Python dictionaries and needs no real files on the host computer.

- **Data representation:** A folder is a dict. Its keys are the names of the items inside it. A file is a str. Its value is the text content of the file.
- **Root:** The root of the whole file system is fs_root.
- **Current directory:** Stored in cwd as a list of folder names. An empty list means the root. For example, ["docs", "school"] means \docs\school.
- **Finding the current directory:** _current_dir() starts at fs_root and walks down through each name in cwd, returning the dictionary for the folder the user is currently in. Every file command calls this helper first and then works on the dictionary it returns.

File Commands

Command	What it does
dir	Prints Directory of \path, then every item in alphabetical order. Folders show ; files show their size in bytes (the length of the string). An empty folder shows (empty).
mkdir	Adds a new empty dictionary under the given name.
create	Adds a new empty string ("") under the given name.
write	Replaces the file's content with the remaining words joined by single spaces. The file must already exist.
read	Prints the file's content, or (empty file) if it has none.
delete	Removes a file or an empty folder.
cd / cd ..	Appends a folder name to cwd, or removes the last name to go up one level.

Rules Enforced by the File System

- mkdir and create refuse a name that already exists in the current folder.
- write and read only work on files; using them on a folder or a missing name is an error.
- delete refuses to remove a folder that still contains items, which prevents accidental loss of data.
- cd only moves into existing folders, and cd .. at the root does nothing.

Limitations:

- Data is not saved: all files disappear when MiniOS exits.
- Names cannot contain spaces, because commands are split on whitespace.
- cd moves one level at a time (no multi-part paths such as cd a/b).
- Multiple spaces inside write text are collapsed to one space.

7. Process Management

Processes in MiniOS are simulated. No real programs are launched and no CPU scheduling takes place. A "process" is a record in a table that represents a program the user has started.

- **The process table:** The table is a dictionary whose key is the PID (process ID) and whose value is a small dictionary holding the process name and status. next_pid is a counter that gives each new process a unique ID.

Commands

- **run** : Checks that a program name was given. Takes the current value of next_pid as the new PID and increases the counter by one. Stores {"name": , "status": "RUNNING"} in the table under that PID. Prints the PID, name and status. The program name is only a label, so any word can be used (for example run notepad).
- **process**: Prints a formatted table of all processes with the columns PID, NAME, and STATUS. If the table is empty it prints No running processes.
- **kill** : Validates that the PID is a number, checks that it exists in the table, and then deletes the entry. This simulates terminating the process.

Behavior notes: PIDs start at 1 and always increase. A PID is never reused, even after its process is killed, just like the unique identifiers in many real systems. Every process has the status RUNNING; there are no other states such as waiting or stopped. Processes are independent of the memory module: starting a process does not allocate memory. Processes are lost when MiniOS exits.

8. Memory Management

Memory is simulated with simple counters measured in megabytes (MB). No real RAM is reserved; the module only tracks how much of a virtual 1024 MB would be in use.

Memory model

Value	Meaning
TOTAL	Total virtual memory: 1024 MB
OS_RESERVED	Fixed amount always taken by the operating system: 256 MB
allocated	Memory the user has requested so far (starts at 0)

The helper _used() returns OS_RESERVED + allocated. Free memory is TOTAL - _used(). At start-up MiniOS therefore shows 256 MB used and 768 MB free, so the user can allocate at most 768 MB.

Commands

- **memory**: Displays the total, used and free memory.
- **allocate** : Adds the amount to allocated, but only if the amount is no larger than the free memory. Otherwise it reports that there is not enough free memory.
- **free** : Subtracts the amount from allocated, but only if the user is not trying to release more than they have allocated. The OS-reserved 256 MB can never be released.

Input validation: Both allocate and free use the shared helper _get_amount(). It prints a usage message if no number was typed and rejects anything that is not a positive whole number (letters, decimals, zero, negatives). It returns the integer, or None if the input was invalid, which tells the calling command to stop.

Limitations: Memory is a single global counter. Allocations are not linked to a specific process, there are no addresses, paging or fragmentation, and the state resets when the program closes.

9. User Management

User management is handled by users.py. It controls who may enter the system and remembers who is currently logged in.

User accounts

Each account is an entry in the USERS dictionary. The key is the username, and the value stores the password and a role. Two accounts are built in:

Username	Password	Role
admin	admin123	Administrator
student	student123	Standard User

The login process

A login screen is displayed. The user is given 3 attempts. Each attempt asks for a username (using input) and a password (using getpass, so the typed password is not shown on screen).

- If the username exists and the password matches, current_user is set, a welcome message is shown, and login() returns True.
- If not, the attempt counter drops by one and the remaining attempts are shown.
- After three failures the message Too many failed attempts. System locked. is printed and login() returns False. In main.py, a False result makes main() return immediately, so the shell never starts and an unauthorised user cannot reach any command.

Use of the logged-in user

The whoami command prints users.current_user, and systeminfo includes the user in its report.

Limitations: Passwords are stored as plain text in the source code (acceptable for a classroom simulation, but not for a real system). The role field is stored but not yet used to restrict any command, so both accounts currently have the same permissions. There is no command for adding users or changing passwords, and no logout (the user exits MiniOS and starts it again).

10. Error Handling

MiniOS is designed so that wrong input never crashes the program. Errors are handled at several levels.

Shell-level handling (main.py)

- Empty input is ignored and the prompt is shown again.
- Unknown commands are caught by checking whether the name exists in COMMANDS. The system prints Unknown command. and suggests typing help.
- Unexpected exceptions: Every command call is wrapped in try / except Exception, which prints Error: and returns to the prompt instead of terminating the program.
- Ctrl+C and Ctrl+D: Catching KeyboardInterrupt and EOFError around input() treats them as an exit command, so MiniOS shuts down cleanly rather than showing a traceback.

Command-level validation

Each command checks its input before doing any work and prints a clear message, then returns without changing any data.

Area	Check	Message
All commands needing arguments	Argument missing	A usage line such as Usage: mkdir
File system	Name already exists (mkdir, create)	Error: Name already exists.
File system	File missing or is a folder (write, read)	Error: File does not exist.
File system	Item missing (delete)	Error: File or directory does not exist.
File system	Folder not empty (delete)	Error: Directory is not empty.
File system	Folder missing or is a file (cd)	Error: Directory does not exist.
Processes	PID is not numeric (kill)	Error: PID must be a number.

Processes	PID not in table (kill)	Error: Process does not exist.
Memory	Amount not a positive whole number	Error: Amount must be a positive number.
Memory	Request larger than free memory (allocate)	Error: Not enough free memory.
Memory	Release larger than allocated (free)	Error: Cannot free more memory than allocated.
Login	Wrong username or password	Error: Invalid username or password. Attempts left: n
Login	Three failed attempts	Too many failed attempts. System locked.

Design approach

- **Validate first, change later.** Data is only modified after every check has passed, so a failed command never leaves the system in a half-changed state.
- **Consistent messages.** Errors begin with Error: which makes problems easy to recognise, and usage hints begin with Usage:.
- **Fail safely.** Because the shell catches any exception that slips past the validation, a bug in one command cannot bring down the whole system.

11. Testing

The MiniOS Omega operating system was tested by evaluating the functionality of its seven main components, which include the command line interface, file management, process management, memory management, user management, system information, and error handling. The functionality of the command line interface was tested by inputting valid and invalid commands into the system to see whether the responses would be accurate. File management was tested by implementing the different operations that the system could perform on files. Process management was tested by simulating the creation, monitoring, and termination of processes. Memory management was tested by simulating memory allocation and memory deallocation. The user management component was tested by inputting valid and invalid logins to test the system authentication mechanism. System information was tested to check whether the right system details are shown, while error handling was tested by inputting invalid inputs and unsupported commands to see whether the proper error message is displayed

12. Problems Encountered

While developing MiniOS Omega, there have been various challenges that the group faced with respect to implementation and integration of the operating system features such as command line interface, file management, process management, memory management, user management, system information, and error handling. The main challenge we have had is in making sure that the command line interface is capable of recognizing user commands and responding to it accordingly. This was sorted out by going through the code, examining the command conditions, and testing different inputs. Another challenge was in the file management operation where there was need for handling the files well. The solution for this challenge was going through the file handling logic and testing the supported operations individually. There has also been challenge in simulation of process and memory management because this feature requires the program to simulate the process operation and memory resource allocation and release. These challenges have been sorted out by reviewing the program logic and testing various scenarios and getting the results. In user management, there has been challenge in making sure that login credentials are validated well and users are only allowed to continue with further steps after logging in successfully. Another challenge was in improving our error handling system for the purpose of providing meaningful messages to the users in case they entered wrong commands or carried out operations that were not supported. Another challenge has been integration of all the features in one program and to avoid possible conflict between the various features. The challenge has been sorted out by distributing tasks among members, working together, reviewing each other's work, and testing the whole program.

13. Conclusion

In conclusion, MiniOS Omega was developed to demonstrate the basic functions of an operating system. The features help users understand how an operating system works. During the development process, our group encountered several challenges. We solved these problems through teamwork, debugging, and repeated testing. Each group member contributed ideas and efforts to complete the project. We tested every feature to check whether it worked as expected. We also identified errors and made improvements to the system. Through this experience, we improved our programming and problem-solving skills. We learned how important proper planning and communication are in software development. The project also helped us understand how different system components work together. We gained practical experience in writing code, checking program outputs, and correcting mistakes. We also learned the importance of making sure that each feature works properly before combining it with the others. Working on MiniOS Omega helped us become more responsible and cooperative as a group. This project was a valuable learning experience that strengthened our knowledge of programming and gave us a better understanding of operating system development.